线性回归和逻辑回归实验

姓名　顾凯琪

学号　226001624

学院 信息学院

专业班级 计算机 2 班

一：实验目的

1. 学习掌握相关理论知识，包括线性回归、逻辑回归的基本原理，模型训练方法等，并通过编程，从实践的角度深化模型的理解；

2. 学习使用梯度下降法进行模型训练的基本原理，并自行使用代码进行梯度下降的实现，将其运用到线性回归、逻辑回归模型的训练；

3. 理解学习率、迭代次数等参数的含义，并通过实验探究其对模型训练过程产生的影响；

4. 通过绘图进行训练过程、模型预测效果的可视化，并通过均方误差等指标对模型的性能进行衡量。

二：实验环境

操作系统：

Windows 11 Version 22H2

集成开发环境：

1. Anaconda3 with Jupyter Notebook
2. PyCharm

相关 Python 库：

1. numpy
2. matplotlib（3.8.3）
3. sklearn（scikit-learn1.4.1）
4. pandas（2.2.1）

三：实验内容

1. 用 sklearn 框架利用线性回归实现预测西瓜成熟度任务；
2. 用 sklearn 框架利用逻辑回归实现分类西瓜好坏任务；

3. 用 python(自己实现，不用框架)实现梯度下降法求解 2 中任务。（选做）

四：实验步骤

1. 实验准备

在第一次实验中已经完成了 conda 虚拟环境搭建，故本次实验基于该虚拟环境进行。对于实验所用数据，使用实验指导给定的西瓜数据，分别用于回归问题和分类问题。数据内容如图 4 所示。

编号	密度	含糖率	好瓜
1	0.697	0.46	1
2	0.774	0.376	1
3	0.634	0.264	1
4	0.608	0.318	1
5	0.556	0.215	1
6	0.403	0.237	1
7	0.481	0.149	1
8	0.437	0.211	1
9	0.666	0.091	0
10	0.243	0.0267	0
11	0.245	0.057	0
12	0.343	0.099	0
13	0.639	0.161	0
14	0.657	0.198	0
15	0.36	0.37	0
16	0.593	0.042	0
17	0.719	0.103	0

编号	密度	含糖量	成熟度
1	0.697	0.46	0.89
2	0.774	0.376	0.9
3	0.634	0.264	0.76
4	0.608	0.318	0.78
5	0.556	0.215	0.58
6	0.403	0.237	0.61
7	0.481	0.149	0.6
8	0.666	0.091	0.48
9	0.243	0.268	0.42
10	0.245	0.057	0.33
11	0.343	0.099	0.37
12	0.639	0.161	0.45
13	0.657	0.198	0.4
14	0.36	0.37	0.22

图 4 实验数据（左-回归问题，右-分类问题）

2. 使用 sklearn 库实现模型训练

（1）数据预处理

给定的数据存于 excel 表格当中，使用 pandas 工具进行读入，并将数据整合成标准形状的 numpy 数组，便于后续的模型训练，其中数据特征的形状应当为[n,2]，数据标签应为[n,1]，具体的处理方法详见代码。

（2）训练、测试集划分

使用 train_test_split()函数进行数据集的划分，该函数进行数据集的随机划分且保证划分结果互斥。其中设定测试集的数据占比 10%。

（3）模型训练

对于 sklearn 库，已经完成了基本的模型训练过程封装，只需要进行模型的实例化，调用 fit()函数传入训练数据即可完成训练，调用 predict 函数并传入

测试数据的特征即可得到对应的结果。

其中线性回归模型为 LinearRegression()，分类问题对应的逻辑回归模型为 LogisticRegression()。

（4）训练结果查看

为查看模型的训练效果，对训练数据的原始标签与模型输出标签进行绘图呈现。数值方面，使用均方误差这一指标进行衡量。同时，也可以访问对象的 coef_ 属性查看模型对应的参数权重。

2. 使用 sklearn 库实现模型训练

（1）损失函数定义

机器学习模型训练的本质是通过调整模型参数使得模型输出结果与真实结果之间的差异最小，而衡量差异通常使用损失函数（Loss Function），对于回归问题，损失函数通常为均方误差，即所有样本数据的真实标签与模型输出标签之间的差值平方之和，表达式如下：

$$Loss(\theta)=\frac{1}{2m}\sum_{i=1}^{m}(y^{(i)}-f_\theta(x^{(i)}))^2$$

对于逻辑回归模型，用于二分类问题，其使用对数损失函数，也称为交叉熵损失函数，表达式如下，其中模型 f 的输出经过 sigmoid 函数进行约束。

$$Loss(\theta)=-\sum_{i=1}^{m}y^i\log(f_\theta(x^i)+(1-y^i)\log(1-f_\theta(x^{(i)})$$

使用 Python 代码分别实现以上损失函数。

（2）定义训练函数

进行训练函数的定义，该函数使用梯度下降法进行模型的训练，函数内需要进行多轮迭代，每次迭代中，需要以模型参数为自变量，对损失函数进行求导，得到当前时刻的梯度值，该梯度值指示了损失函数变化最快的方向，将模型参数往该方向进行更新，更新的步长由学习率进行约束。形式化地表达如下：对于每个参数θ_j，调整过程如下式所示，其中 α 为学习率：

$$\theta_j:=\theta_j-\alpha\frac{\partial}{\partial\theta_j}Loss(\theta)$$

具体地，该函数需要传入以下内容：模型参数、训练集特征、训练集标签、

迭代次数、学习率，通过函数内的迭代，模型参数不断调整，过程中可以记录损失函数以查看模型的训练过程。

（3）模型的训练与验证

该步骤与使用 sklearn 库进行模型训练验证的步骤一致，只是将模型的训练转移到了自行编写的基于梯度下降的训练函数中。最后需要进行训练结果的查看与验证。

3. 学习率调整

对模型的学习率与迭代次数进行调整，记录损失函数的变化过程，分析学习率与迭代次数对模型训练的具体影响。

五：代码 （头文件和数据集部分代码相同，只展示一次）

```
"""
--------头文件导入--------
"""
```

```
1. import sklearn
2. import matplotlib.pyplot as plt
3. from sklearn import model_selection, linear_model, metrics
4. import pandas as pd
5. import numpy as np
```

```
"""
--------数据集导入和划分--------
"""
```

```
1. # 读取数据
2. excel_name = "./西瓜数据集 2.xlsx"
3. data = pd.read_excel(excel_name)
4.
5. # 提取特征和目标变量
6. x = data.iloc[:, 1:3].values
7. y = data.iloc[:, 3].values
8.
9. # 拆分训练集和测试集
10.x_train, x_test, y_train, y_test = model_selection.train_test_spl
```

```
    it(x, y, test_size=0.1, shuffle=True)
```

linear_model.py（线性回归）：

```
"""
--------训练--------
"""
```

```
1. # 训练线性回归模型
2. L = linear_model.LinearRegression()
3. L.fit(x_train, y_train)
4.
5. # 预测训练集
6. y_train_predict = L.predict(x_train)
```

```
"""
--------可视化--------
"""
```

```
1. # 绘图
2. plt.figure(figsize=(8, 5))  # 调整画布大小
3. plt.plot(range(len(y_train)), y_train, 'b-', label="true")  # 蓝色
   实线
4. plt.plot(range(len(y_train_predict)), y_train_predict, 'r--', lab
   el="predict")  # 红色虚线
5.
6. # 添加标题、图例、坐标轴标签
7. plt.xlabel("sample")
8. plt.ylabel("values")
9. plt.title("train")
10.plt.legend()
11.plt.grid(True)  # 添加网格线
12.plt.show()
```

```
"""
--------预测和分析--------
"""
```

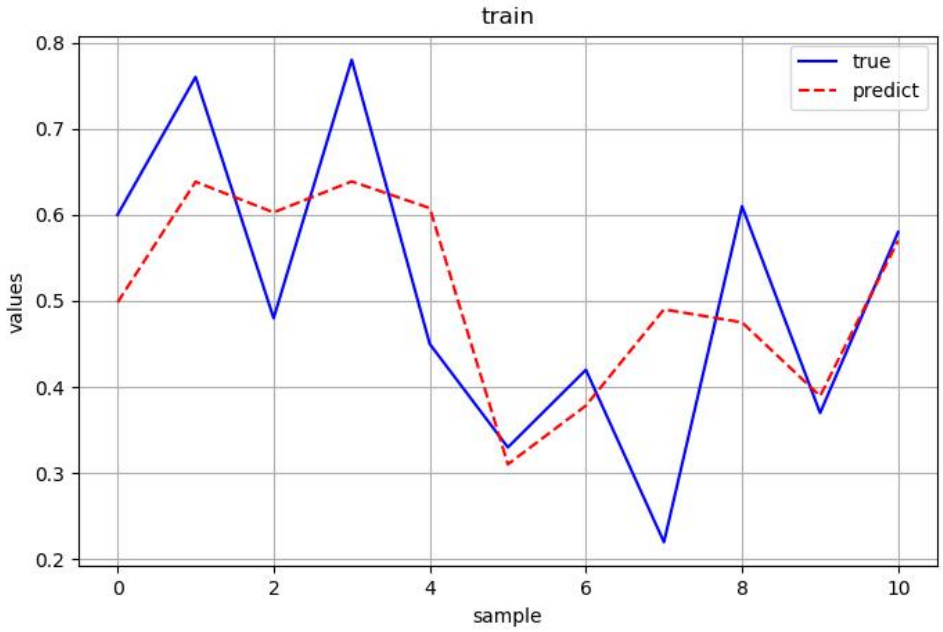
```
1. # 测试集预测
```

```
y_test_predict = L.predict(x_test)
print(y_test_predict)
print(y_test)

# 计算误差
err = metrics.mean_squared_error(y_test_predict, y_test)
print(f"测试集损失为: {err}")
print("模型系数:", L.coef_)

```

使用 sklearn 库进行线性回归模型的训练，在训练集上的训练结果如下图所示。在测试集上的均方误差为 0.0355，模型所给出的系数比为[0.6680,0.3292]，由此可以看出含糖量对于最终成熟度的影响最大。



图一 线性回归拟合曲线

```
D:\Anaconda\python.exe "C:\Users\asus\Desktop\x\试卷卷卷子\机器学习\实验\实验课2 线性回归_西瓜成熟度线性模型预测及分类\linear.py"
[0.63200912 0.76877032]
[0.4 0.9]
测试集损失为: 0.035524730998727 05
模型系数: [0.66807669 0.32919226]

进程已结束，退出代码为 0
```

图二 线性回归偏差和性能分析

logistic_regression.py:

```
"""
--------模型训练和预测--------
"""
```

```python
# 使用逻辑回归模型
L = LogisticRegression(max_iter=200)
L.fit(x_train, y_train)

# 预测测试集
y_predict = L.predict(x_test)

# 输出预测结果与真实结果
print("预测结果:", y_predict)
print("真实结果:", y_test)

# 计算并输出准确率
accuracy = metrics.accuracy_score(y_test, y_predict)
print(f"准确率: {accuracy}")

# 计算并输出交叉熵损失
log_loss = metrics.log_loss(y_test, L.predict_proba(x_test))
print(f"交叉熵损失: {log_loss}")

# 输出模型系数
print("模型系数:", L.coef_)

```

对于逻辑回归，考虑计算分类的准确率，代码部分如下图所示。可以看到模型对于测试集的准确率为 0.5，使用交叉验证得到的损失值为 0.6912，得到的模型系数为[-0.032，0.2492]。

```
D:\Anaconda\python.exe "C:\Users\asus\Desktop\x\试卷卷卷子\机器学习\实验\实验课2 线性回归_西瓜成熟度线性模型预测及分类\log.py"
预测结果: [0 0]
真实结果: [1 0]
准确率: 0.5
交叉熵损失: 0.6912028870989696
模型系数: [[-0.03199497  0.24919741]]
```

图三 逻辑回归偏差和性能分析

sgd_logistic_regression.py:

```
"""
--------头文件导入--------
"""
```

```
import numpy as np
import pandas as pd
from sklearn import model_selection
from sklearn.linear_model import LogisticRegression
```

```
"""
--------逻辑回归函数--------
"""
```

```
class LogisticRegressionGD:
    def __init__(self, learning_rate=0.01, n_iterations=1000):
        self.learning_rate = learning_rate  # 学习率
        self.n_iterations = n_iterations    # 迭代次数
        self.weights = None                 # 权重
        self.bias = None                    # 偏置

    def sigmoid(self, z):
        # Sigmoid 函数
        return 1 / (1 + np.exp(-z))

    def compute_loss(self, X, y):
        # 计算交叉熵损失
        n_samples = len(y)
        y_predicted = self.sigmoid(np.dot(X, self.weights) + self.bias)
        # 防止对数运算中的log(0)出现
        epsilon = 1e-15
        y_predicted = np.clip(y_predicted, epsilon, 1 - epsilon)
        loss = -(1 / n_samples) * np.sum(y * np.log(y_predicted) + (1 - y) * np.log(1 - y_predicted))
        return loss
```

```
21.
22.     def fit(self, X, y):
23.         n_samples, n_features = X.shape           # 样本数量和特征数
    量
24.         self.weights = np.zeros(n_features)       # 初始化权重
25.         self.bias = 0                             # 初始化偏置
26.
27.         # 梯度下降
28.         for i in range(self.n_iterations):
29.             linear_model = np.dot(X, self.weights) + self.bias
          # 计算线性模型
30.             y_predicted = self.sigmoid(linear_model)
          # 计算预测值
31.
32.             # 计算梯度
33.             dw = (1 / n_samples) * np.dot(X.T, (y_predicted - y))
          # 计算权重的梯度
34.             db = (1 / n_samples) * np.sum(y_predicted - y)
          # 计算偏置的梯度
35.
36.             # 更新参数
37.             self.weights -= self.learning_rate * dw
          # 更新权重
38.             self.bias -= self.learning_rate * db
          # 更新偏置
39.
40.             # 每隔一定次数输出损失
41.             if i % 100 == 0:
42.                 loss = self.compute_loss(X, y)
43.                 print(f"Iteration {i}, Loss: {loss}")
44.
45.     def predict(self, X):
46.         linear_model = np.dot(X, self.weights) + self.bias
        # 计算线性模型
47.         y_predicted = self.sigmoid(linear_model)
        # 计算预测值
48.         y_predicted_cls = [1 if i > 0.5 else 0 for i in y_predict
    ed]   # 将概率值转换为类别
49.         return np.array(y_predicted_cls)
        # 返回预测类别
```

"""

```
--------主函数（数据导入和训练预测）--------
"""
```

```python
if __name__ == '__main__':
    # 读取数据
    excel_name = "西瓜数据集.xlsx"
    data = pd.read_excel(excel_name)

    # 提取特征和目标变量
    x = data[data.columns[1:3]].values
    y = data[data.columns[3]].values

    # 拆分训练集和测试集
    x_train, x_test, y_train, y_test = model_selection.train_test_split(x, y, test_size=0.1, shuffle=True)

    # 创建模型并训练
    model = LogisticRegressionGD(learning_rate=0.01, n_iterations=1000)
    model.fit(x_train, y_train)

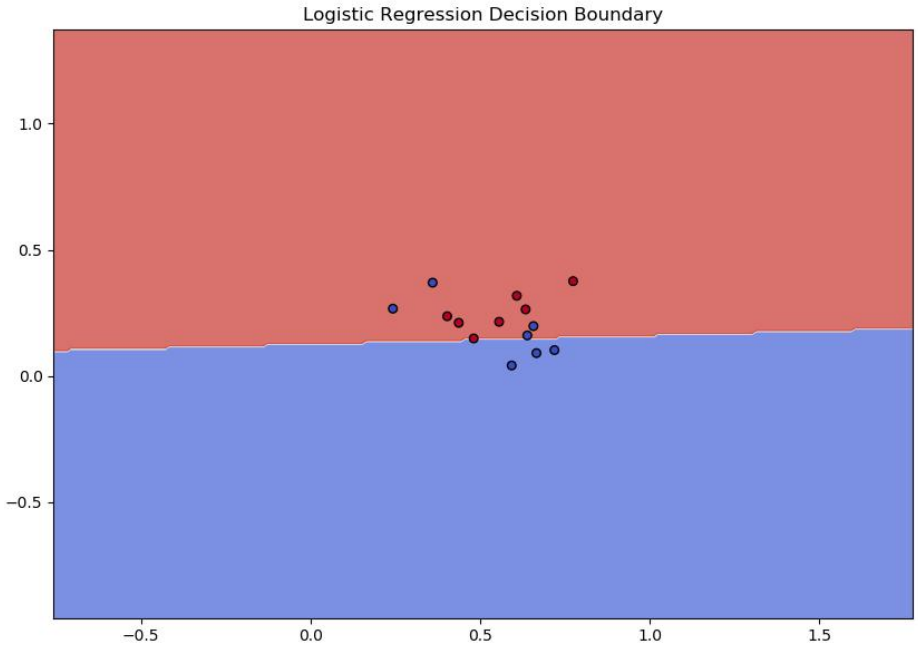
    # 进行预测
    predictions = model.predict(x_test)
    print("预测结果:", predictions)
    print("真实结果:", y_test)

    # 输出交叉熵损失
    final_loss = model.compute_loss(x_test, y_test)
    print(f"最终交叉熵损失: {final_loss}")

    # 输出模型系数
    print("模型系数 (权重):", model.weights)
    print("模型偏置:", model.bias)
```

使用梯度下降法实现模型的训练过程，代码上所述，在迭代训练过程中记录 Loss 的变化，以描述训练的过程。

将自定义的训练函数运用于线性回归模型，首先随机初始参数值，设置学习率为 0.01，迭代次数为 1000，Loss 变化趋势和预测分布如图所示。



图四 逻辑回归（梯度下降）预测结果

最终的交错损失值为 0.6877254492046498，这是模型在训练结束后的损失值，反映了模型预测的误差。

模型系数为[-0.0064223，0.18491602]，这是梯度下降中特征的权重。

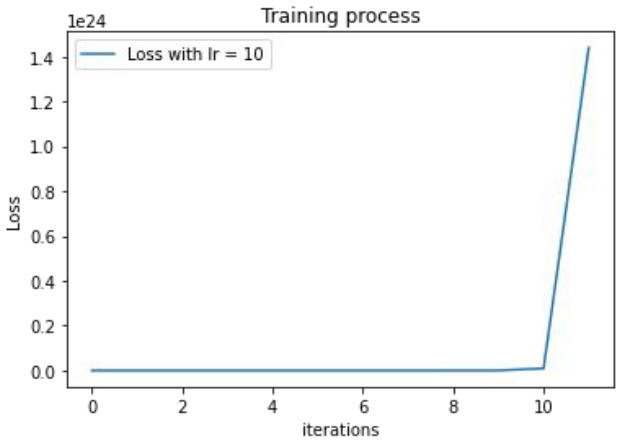
模型偏置为-0.023411513602655767，这是梯度下降中的截距项，用于更改好的拟合回归函数。

```
D:\Anaconda\python.exe "C:\Users\asus\Desktop\x\试卷卷卷子\机器学习\实验\实验课2 线性回归_西瓜成熟度线性模型预测及分类\梯度下降.py"
Iteration 0, Loss: 0.693143485695572
Iteration 100, Loss: 0.6927788658005639
Iteration 200, Loss: 0.6924211017737605
Iteration 300, Loss: 0.6920676353214944
Iteration 400, Loss: 0.6917171662424738
Iteration 500, Loss: 0.6913690314533089
Iteration 600, Loss: 0.6910228905294068
Iteration 700, Loss: 0.6906785664660857
Iteration 800, Loss: 0.690335965041728
Iteration 900, Loss: 0.6899950339839983
预测结果: [0 0]
真实结果: [0 0]
最终交叉熵损失: 0.6877254492046498
模型系数（权重）: [-0.0064223   0.18491602]
模型偏置: -0.023411513602655767

进程已结束，退出代码为 0
```

图五 逻辑回归（梯度下降）偏差和性能分析

通过理论学习可知，当学习率设置过大时，可能会导致 Loss 跳跃过多，从而错过 Loss 函数和的最低点，而导致模型训练不收敛，如图 6.6 所示，将学习率设置为 10，发现 Loss 最后不减反增。



图五 设置学习率为 10 的得到的 Loss 变化

当学习率设置过小时，会减缓模型的收敛速度，若没有足够多的迭代次数，则会导致无法到达较优的参数中，另一方面也增加了模型的训练时长。

六：问答题

在判断西瓜成熟度这个问题上，请解释密度跟含糖量哪个指标更重要？

回答：从实验结果可以看出，密度的系数（0.6680）大于含糖量的系数（0.3292），因此密度对西瓜成熟度的影响更大，是判断西瓜成熟度更重要的指标。

依据：根据线性回归模型的训练结果，模型给出的系数比为[0.6680，0.3292]，其中密度的系数为 0.6680，含糖量的系数为 0.3292。系数的大小反映了特征对目标变量（西瓜成熟度）的影响程度，系数越大，说明该特征对目标变量的影响越大。